

Análisis Espaciotemporal de Siniestralidad Vial a Gran Escala y Benchmarking Experimental de Arquitecturas Mononodo vs. Distribuidas (Pandas vs. Apache Spark)

MIA - Grupo 4 | Maestría en Inteligencia Artificial

Facultad de Ingeniería — Universidad de Palermo (UP)

Asignatura: Big Data | Buenos Aires, Argentina

Repositorio oficial y código reproducible: <https://github.com/jdiegoweber-debug/BIG-DATA-GRUPO-4.git>

Resumen

El presente trabajo aborda el procesamiento analítico y modelado predictivo a gran escala del dataset oficial US Accidents (2016–2023), compuesto por 7,728,394 registros viales continuos y 46 atributos heterogéneos (~2.85 GB en CSV plano). Frente a los desafíos de saturación de memoria RAM física y latencia en entornos mononodo, se implementó una estrategia de ingeniería de datos basada en el formato columnar Apache Parquet con compresión ZSTD, ordenamiento físico espacial por bloques y codificación por diccionario, reduciendo el almacenamiento a 491.33 MB (~83% de ahorro físico de almacenamiento y lecturas de I/O). Se ejecutó un benchmarking experimental riguroso contrastando la ejecución mononodo vectorizada en Pandas versus la arquitectura distribuida en Apache Spark (PySpark 3.5.0) para las cinco consultas analíticas asignadas al Grupo 4: C4 (Hotspots críticos evaluando doble enfoque: Severidad Crítica Nacional vs Densidad Absoluta Estatal), C3 (Búsqueda geoespacial Haversine con validación paramétrica cruzada en Los Ángeles y Nueva York, desglosando tiempos y sobrecarga basal de la JVM), C6 (Ranking de autopistas críticas normalizadas evaluando tanto la Tasa Relativa de Severidad como el Volumen Absoluto), C8 (Detección de anomalías climáticas extremas mediante doble perspectiva: impacto volumétrico y rareza por Z-Score con variables meteorológicas), y C10 (Velocidad de despeje según iluminación solar y crepúsculo astronómico contrastando días hábiles con fines de semana mediante matriz de 8 cuadrantes). Asimismo, se diseñó y entrenó un pipeline distribuido de Machine Learning con Spark MLlib (DecisionTreeClassifier) para la clasificación supervisada de severidad vial, alcanzando una Exactitud Global (Accuracy) del 79.81% y un F1-Score ponderado de 0.7138. Los resultados cuantitativos demuestran que, si bien Pandas supera a Spark en operaciones mononodo acotadas debido al overhead de la JVM y la serialización Py4J, Apache Spark resulta insustituible para transformaciones complejas con particionado, funciones de ventana y escalabilidad distribuida out-of-core ante volúmenes masivos.

Palabras Clave: Big Data, Apache Spark, PySpark, Pandas, Benchmarking, Siniestralidad Vial, Apache Parquet, Spark MLlib, Haversine, Predicate Pushdown.

1. Introducción

La optimización de la seguridad vial y la mitigación de siniestros en redes de transporte de escala continental constituyen un problema neurálgico para las agencias gubernamentales, aseguradoras y planificadores urbanos. La proliferación de redes telemáticas, sensores viales y estaciones meteorológicas conectadas en tiempo real ha dado lugar a volúmenes masivos de datos continuos que documentan cada incidente con un nivel de detalle milimétrico. No obstante, procesar, analizar y extraer patrones predictivos a partir de estos repositorios impone severas restricciones a los sistemas analíticos tradicionales.

El dataset oficial US Accidents (2016–2023), que concentra más de 7.7 millones de siniestros viales registrados en Estados Unidos a lo largo de 49 estados, ejemplifica la problemática fundamental del Big Data: la incapacidad de las herramientas analíticas mononodo estándar para manipular y transformar datos interactivos sin incurrir en desbordamientos de memoria física (Out-Of-Memory, OOM) o degradaciones extremas de latencia.

En este marco, el presente proyecto integrador de la Maestría en Inteligencia Artificial de la Universidad de Palermo tiene como propósito evaluar experimentalmente dos paradigmas computacionales complementarios: la computación vectorizada mononodo en memoria RAM mediante Pandas (con librerías de bajo nivel en C/C++) frente al paradigma distribuido basado en Apache Spark (PySpark 3.5.0), gobernado por el optimizador lógico/físico Catalyst y el gestor de memoria Tungsten. A partir de las cinco consultas analíticas asignadas al Grupo 4 y el entrenamiento de un pipeline de Machine Learning a escala, se contrastan los tiempos de respuesta netos (wall-clock time), los factores de aceleración (Speedup) y los cuellos de botella computacionales de cada arquitectura.

2. Desarrollo Metodológico

2.1. Datos y Preprocesamiento

El dataset analizado corresponde al compendio de accidentes viales de Estados Unidos provisto por Sobhan Moosavi et al. en Kaggle. El archivo fuente original en

formato CSV plano posee un tamaño de 2.85 GB (3,058,183,727 bytes) y alberga 7,728,394 tuplas con 46 variables. Estas comprenden coordenadas geográficas de inicio y fin (Start_Lat, Start_Lng, End_Lat, End_Lng), marcas temporales de inicio y fin (Start_Time, End_Time), clasificación de gravedad del impacto vial (Severity de 1 a 4), variables meteorológicas continuas (Temperature, Visibility, Humidity, Pressure, Wind_Speed, Precipitation) y variables categóricas contextuales (State, Street, Sunrise_Sunset).

La conversión a Apache Parquet se ejecutó en streaming por bloques de 250,000 filas empleando PyArrow con esquema tipado estricto. Se aplicaron tres optimizaciones físicas: (1) Compresión ZSTD, (2) Codificación por diccionario para strings frecuentes (Street, State), y (3) Ordenamiento físico interno por bloque según Start_Lat y Start_Lng con null_placement='at_end', preservando los 7,728,394 registros completos. Esta estructura redujo el volumen a 491.33 MB (83% de ahorro), habilitando Predicate Pushdown y Projection Pushdown en ambas plataformas.

2.2. Entorno de Cómputo Experimental

- Estación Local: Intel Core i5-1135G7 (4 núcleos físicos, 8 hilos @ 2.40 GHz base, turbo 4.20 GHz), 16 GB RAM DDR4 (15.75 GB visibles), SSD NVMe PCIe M.2, Windows 11 Home 64-bit, Python 3.12.0, PySpark 3.5.0 master('local[*]'), Pandas 2.2+, PyArrow 14.0+ y OpenJDK 17 LTS.

- Cloud Colab: VM Linux Ubuntu 22.04 LTS x86_64, 2 vCPUs Intel Xeon @ 2.20 GHz, 12.7 GB RAM disponible, SSD de 100 GB y PySpark 3.5.0.

2.3. Acceso al Código y Portabilidad Multi-Entorno

El notebook interactivo BIG-DATA-GRUPO-4-Colab.ipynb incorpora introspección de entorno ('google.colab' in sys.modules) para ejecutar de punta a punta sin fallas en Colab o en local. Código público en GitHub: <https://github.com/jdiegoweber-debug/BIG-DATA-GRUPO-4.git>.

3. Resultados Analíticos de las Consultas Asignadas

Conforme a la matriz oficial de asignación del proyecto integrador (Página 2 del enunciado), al Grupo 4 le correspondió responder la Consulta 4 (obligatoria para todos los grupos) junto con el lote asignado compuesto por las Consultas 3, 6, 8 y 10.

3.1. Consulta 4: Hotspots Espaciales Críticos — Doble Enfoque Metodológico y Escala Territorial

Consigna Oficial y Definición de Criterios: La consigna exige reportar las 5 coordenadas geográficas más peligrosas (a nivel nacional o en un estado de alto flujo

como California) dado un radio de influencia ($R = 2.0$ km o 5.0 km), requiriendo definir formalmente el criterio de peligrosidad y el método espacial. Para abordar integralmente la problemática vial, el Grupo 4 implementó y comparó los dos criterios previstos por la cátedra:

1. Enfoque A — Concentración de Severidad Crítica (Severity ≥ 3) a Nivel Nacional ($R \approx 2.0$ km): Diseñado para identificar los puntos donde los accidentes conllevan riesgo letal, lesiones graves y parálisis de la red federal. Se discretizó el territorio en micro-celdas mediante redondeo a 2 decimales (round(2), paso de $0.01^\circ \approx 1.1$ km). El ranking nacional identificó:

- Rank 1: Lat 33.94, Lng -118.10 (1,619 accidentes graves) — Downey / Los Ángeles, CA (Intercambiador crítico I-105 y I-710).

- Rank 2: Lat 40.85, Lng -73.94 (1,550 accidentes graves) — George Washington Bridge / Cross Bronx Expressway (I-95), NY/NJ (el cuello de botella más accidentado del Corredor Este).

- Rank 3: Lat 33.77, Lng -84.39 (1,432 accidentes graves) — Atlanta, GA (Downtown Connector I-75 / I-85 Centro).

- Rank 4: Lat 33.74, Lng -84.39 (1,372 accidentes graves) — Atlanta, GA (Downtown Connector I-75 / I-85 Sur).

- Rank 5: Lat 33.79, Lng -84.39 (1,301 accidentes graves) — Atlanta, GA (Downtown Connector I-75 / I-85 Norte).

2. Enfoque B — Densidad Absoluta Total en California ($R \approx 5.0$ km): Diseñado para dimensionar la congestión estructural y el volumen acumulado de siniestros totales (todas las severidades) en el estado con mayor parque automotor. Se empleó una macro-grilla con paso de 0.10° (~ 11.1 km, bins centrados en X.X50, equivalente al radio de 5.0 km). El Top 5 estatal demostró una concentración unívoca en la megalópolis de Los Ángeles:

- Hotspot #1: Lat 34.050, Lng -118.250 (50,254 accidentes) — Downtown Los Ángeles (nudo de máxima fricción vehicular de EE.UU.).

- Hotspot #2: Lat 33.950, Lng -118.250 (36,294 accidentes) — Sur de Los Ángeles / Inglewood / Corredor I-110.

- Hotspot #3: Lat 34.050, Lng -118.150 (29,171 accidentes) — East Los Ángeles / Montebello / Acceso I-5.

- Hotspot #4: Lat 33.850, Lng -118.250 (25,091 accidentes) — Carson / Acceso Corredor Portuario Long Beach.

- Hotspot #5: Lat 33.950, Lng -118.150 (23,903 accidentes) — Compton / Lynwood / Cruce I-105 e I-710.

Discusión y Aplicabilidad Operativa: El Enfoque A es la herramienta óptima para la asignación estratégica de helicópteros médicos, unidades de rescate y señalización de emergencia en arterias federales (I-95, I-75, I-85). En cambio, el Enfoque B proporciona una métrica de capacidad e ingeniería civil urbana, señalando macro-áreas metropolitanas donde las colisiones masivas generan pérdidas millonarias en horas-hombre por congestiónamiento.

3.2. Consulta 3: Búsqueda Espacial Haversine y Validación Paramétrica Bipodal (Pandas vs. Spark)

Consigna y Carácter Paramétrico: Filtrar y ordenar los accidentes dentro de un radio $R = 25.0$ km respecto de un epicentro geográfico (Lat0, Lng0) mediante la distancia esférica de Haversine: $d = 2r \arcsin(\sqrt{\sin^2(\Delta\phi/2) + \cos(\phi_1)\cos(\phi_2)\sin^2(\Delta\lambda/2)})$, con $r = 6371$ km. Dado que el enunciado no impone un único punto geográfico forzoso, el Grupo 4 diseñó un algoritmo de búsqueda paramétrica y ejecutó una validación experimental cruzada en los dos polos metropolitanos de mayor volumen vial de los Estados Unidos: la Costa Oeste (Los Ángeles, CA) y la Costa Este (Nueva York, NY).

Resultados en Los Ángeles, CA (Lat0 = 34.05, Lng0 = -118.24): Se detectaron exactamente 361,038 siniestros en el radio de 25 km. Los eventos más próximos al epicentro se ubicaron a solo 45 metros (Lat: 34.049621, Lng: -118.239820) y 68 metros (Lat: 34.050218, Lng: -118.240689). La consistencia entre Pandas y PySpark fue del 100% en conteo y en el ranking de proximidad. El tiempo de procesamiento fue de 1.67 s en Pandas versus 28.32 s en PySpark.

Resultados en Nueva York, NY (Lat0 = 40.7128, Lng0 = -74.0060): Se detectaron 269,717 siniestros en el radio de 25 km. El accidente más cercano se ubicó a 90 metros (ID A-7522120, Lat: 40.711990, Lng: -74.005990). Asimismo, la consulta identificó un micro-hotspot de siniestros repetidos a exactamente 105 metros (Lat: 40.712474, Lng: -74.004824, registros A-4415744, A-6644883, A-6868432, A-6648478, A-3655810, etc.), confirmando una intersección de recurrencia crítica en Manhattan. El tiempo de ejecución en Pandas ascendió a 8.834 s al proyectar identificadores textuales (ID) y ordenar sin prefiltrado de Bounding Box, frente a 1.67 s del pipeline vectorizado con tipos primitivos puros.

Discrepancia Espacial y Topológica: La diferencia volumétrica entre Los Ángeles (361,038) y Nueva York (269,717) obedece a factores geográficos e infraestructurales: (a) En Manhattan, el radio de 25 km abarca extensos cuerpos de agua (Hudson River, East River y Upper NY Bay), reduciendo la superficie vial

efectiva en ~35%; en contraste, Los Ángeles presenta una cuadrícula terrestre ininterrumpida de autopistas masivas (I-5, I-10, I-110, I-405); y (b) Cobertura sensorial: California concentra la mayor tasa de reporte del dataset nacional (1.74 millones de accidentes acumulados).

Comparativa de Tiempos y Factor de Aceleración (Speedup): El contraste entre las tres implementaciones evaluadas (Pandas vectorizado en 1.67 s, Pandas con cadenas en 8.83 s y PySpark en 28.32 s con Speedup formal de 0.059x) se fundamenta en:

- Projection Pushdown y Memoria Activa (1.67 s): Al leer únicamente las 3 columnas numéricas primitivas (Start_Lat, Start_Lng, Severity), Pandas retiene escasos ~180 MB en RAM y aplica las fórmulas trigonométricas mediante instrucciones vectoriales SIMD (AVX2) de NumPy en C contiguo.

- Sobrecarga de Tipos String en Memoria (8.83 s): Al incluir columnas de texto alfanuméricas como ID, el consumo de memoria se triplica, se degrada la caché L2/L3 y el algoritmo de ordenamiento Timsort penaliza el rendimiento por la comparación de punteros a cadenas de caracteres.

- Sobrecarga de Coordinación y Shuffle en PySpark (28.32 s): PySpark incurre en la latencia fija de la JVM y sockets Py4J (15-20 s), requiriendo además un RangePartitioner Shuffle para ejecutar el .orderBy() global por distancia sobre los 361k registros, lo que explica la ventaja de Pandas en arquitecturas mononodo.

3.3. Consulta 6: Ranking de Autopistas Críticas con Normalización Regex (Doble Perspectiva)

Consigna: Normalizar las vías de la columna Street (ej. I-95, I-5, US-101) para elaborar el Top 10 de autopistas con mayor índice de siniestros graves (Severity ≥ 3) y reportar su tiempo promedio de despeje vial.

Justificación de Normalización: El análisis de la columna Street cruda reveló que autopistas troncales aparecían fragmentadas por sentido de circulación (ej. 'I-95 N' con 31,107 siniestros e 'I-95 S' con 28,776). Para cumplir rigurosamente con la consigna, se aplicó una normalización basada en expresiones regulares $r"(I-\backslash d+|US-\backslash d+)\backslash b"$, aislando el código unificado de la autopista (highway).

Doble Enfoque Analítico: La consigna plantea la ambigüedad conceptual del término 'índice de siniestros graves'. El Grupo 4 analizó e implementó formalmente ambas perspectivas:

1. Perspectiva Relativa (Tasa de Severidad): Define el índice como la probabilidad condicional de que un accidente sea grave: $\text{Indice_Gravedad} = \text{accidentes_graves} / \text{accidentes_totales}$ (con soporte mínimo de 100 accidentes totales para evitar sesgos muestrales). Esta

métrica mide la peligrosidad o letalidad intrínseca de la calzada.

2. Perspectiva Absoluta (Volumen Acumulado Normalizado): Mide el volumen masivo de siniestros graves acumulados a lo largo de toda la autopista troncal unificada. Esta métrica cuantifica el impacto en la congestión de la red y la demanda logística de auxilio vial.

Tabla 1. Top 10 Autopistas Críticas de EE.UU. por Tasa Relativa de Severidad (Índice %)

Autopista (Highway)	Accidentes Totales	Accidentes Graves	Despeje Promedio (min)	Índice de Gravedad
I-72	293	270	86.41 min	0.9215 (92.15%)
I-57	3,580	2,905	166.08 min	0.8115 (81.15%)
I-478	312	248	83.01 min	0.7949 (79.49%)
I-126	450	343	71.74 min	0.7622 (76.22%)
I-176	158	119	92.51 min	0.7532 (75.32%)
I-255	751	560	93.38 min	0.7457 (74.57%)
I-474	142	102	66.59 min	0.7183 (71.83%)
US-77	3,976	2,814	135.47 min	0.7077 (70.77%)
US-209	113	78	129.35 min	0.6903 (69.03%)
I-865	316	215	95.19 min	0.6804 (68.04%)

Tabla 2. Top 10 Autopistas Críticas de EE.UU. por Volumen Absoluto Grave Normalizado

Autopista Troncal	Siniestros Graves (Severity >= 3)	Despeje Promedio (min)	Horas de Corte Promedio
I-95	62,202	95.66 min	1.59 horas
I-10	44,309	1,355.04 min	22.58 horas
I-5	38,954	74.95 min	1.25 horas
I-80	31,137	184.81 min	3.08 horas
I-75	29,478	420.54 min	7.01 horas
I-35	25,206	148.72 min	2.48 horas
I-405	20,284	66.78 min	1.11 horas
I-70	17,023	163.80 min	2.73 horas
I-90	16,234	86.31 min	1.44 horas
I-40	15,066	294.58 min	4.91 horas

Discusión de Hallazgos en Consulta 6: La Tabla 1 demuestra que la Interestatal I-72 exhibe un índice de gravedad récord del 92.15%, seguida de la I-57 con un 81.15% y tiempos de despeje de 166.08 min (~2.8 horas). En contraste, la Tabla 2 revela que al unificar carriles, la Interestatal I-95 concentra el mayor impacto nacional con 62,202 accidentes graves (promediando 95.66 min de corte). Asimismo, se destaca el comportamiento crítico de la Interestatal I-10, cuyos 44,309 accidentes graves demandaron un promedio de 1,355.04 minutos (casi 23 horas de interrupción total), reflejando la complejidad de remoción en zonas interestatales desérticas y remotas.

3.4. Consulta 8: Detección de Anomalías Meteorológicas Severas (> 3 Sigma) — Doble Perspectiva Analítica

Consigna Oficial: Detectar días y estados donde los siniestros superaron en más de 3 desviaciones estándar (> 3 sigma) la media histórica del estado, cruzando con variables climáticas continuas (Precipitation, Wind_Speed, Visibility) para corroborar eventos extremos.

Universo de Días Anómalos Detectados: El filtrado estadístico sobre las 7,728,394 tuplas identificó un total exacto de 1,171 días anómalos a nivel nacional que satisfacen formalmente el criterio $Z = (x - \mu) / \sigma > 3.0$. Para dimensionar el fenómeno tanto a nivel de saturación operativa como de disrupción climática, el Grupo 4 implementó dos perspectivas analíticas complementarias:

1. Perspectiva de Impacto Volumétrico Absoluto (Ordenado por Siniestros Totales en 24h): Enfocada en cuantificar el colapso masivo de la infraestructura de asistencia vial y médica. El Top 5 nacional lo monopoliza California (media histórica de 696.0 accidentes/día), registrando picos catastróficos:

- 2021-01-29: 2,823 siniestros (4.05x la media histórica, $Z = 5.12$ sigma, precipitación acumulada de 0.014 in) — Tormenta invernal con deslizamientos en corredores de montaña.
- 2020-12-28: 2,637 siniestros ($Z = 4.67$ sigma, precipitación de 0.029 in) — Temporal costero y heladas.
- 2022-11-08: 2,444 siniestros ($Z = 4.21$ sigma, precipitación de 0.050 in) — Lluvias torrenciales en el sur de California.
- 2020-04-06: 2,331 siniestros ($Z = 3.93$ sigma, precipitación de 0.042 in) — Temporal de primavera con colapso de autopistas.
- 2022-12-31: 2,259 siniestros ($Z = 3.76$ sigma, precipitación de 0.075 in) — Víspera de Año Nuevo con inundaciones viales.

2. Perspectiva de Rareza Meteorológica Extrema (Ordenado por Z-Score Descendente): Mide la magnitud pura con que una tormenta desestabilizó la distribución de un estado, corroborada por las variables físicas de viento y visibilidad:

- Kansas (KS, 2022-02-18): 316 siniestros con $Z = 23.25$ sigma — Ventisca de invierno ártico con calzada congelada.

- Ohio (OH, 2022-12-23): 964 siniestros con Z = 18.27 sigma — Tormenta invernal Elliott con vientos extremos de 26.18 mph y visibilidad crítica de solo 1.29 millas (condiciones de 'blizzard' / tormenta blanca).
- New Hampshire (NH, 2022-01-05): 101 siniestros con Z = 18.09 sigma — Visibilidad nula de 2.79 millas y nieve.
- Iowa (IA, 2021-02-04): 282 siniestros con Z = 17.95 sigma — Vientos polares sostenidos de 21.37 mph y visibilidad de 4.68 millas.
- New Mexico (NM, 2022-02-02): 93 siniestros con Z = 17.74 sigma — Tormenta de nieve en pasos de montaña con visibilidad de 3.62 millas.

Discusión y Aporte Operacional: La Perspectiva 1 es la métrica rectora para el dimensionamiento logístico de ambulancias, patrullas y centros de trauma en estados con alta concentración poblacional. En contraparte, la Perspectiva 2 es la herramienta analítica fundamental para la meteorología vial y la ingeniería predictiva, permitiendo correlacionar vientos > 25 mph y visibilidad < 2 millas con aumentos de más de 17 desvíos estándar en la accidentalidad de estados del Medio Oeste.

3.5. Consulta 10: Velocidad de Despeje según Iluminación Solar, Crepúsculo y Tipo de Día

Consigna Oficial: Comparar el tiempo medio de despeje vial (End_Time – Start_Time) en condiciones diurnas versus nocturnas (Sunrise_Sunset, Astronomical_Twilight), contrastando días hábiles con fines de semana.

Metodología y Matriz Fotométrica Dimensional: Para evaluar con rigor el impacto lumínico y temporal en la red vial, el Grupo 4 procesó los 7,728,394 registros viales calculando la duración neta en minutos y desagregando el fenómeno en una matriz tridimensional de 8 combinaciones fotométricas (Condición Solar × Crepúsculo Astronómico × Tipo de Día), cuyos resultados consolidados se reportan en la Tabla 4.

Tabla 4. Matriz fotométrica y temporal de velocidad de despeje vial (Consulta 10).

Condición Solar	Crepúsculo	Tipo de Día	Siniestros	Despeje Medio (min)	Despeje Medio (horas)
Day (Día)	Day (Día)	Día Hábil	4,178,092	336.66 min	5.61 h
Day (Día)	Night	Día	4,277	142.54	2.38 h

	(Noche)	Hábil		min	
Night (Noche)	Day (Día)	Día Hábil	830,099	345.65 min	5.76 h
Night (Noche)	Night (Noche)	Día Hábil	889,860	649.94 min	10.83 h
Day (Día)	Day (Día)	Fin de Semana	650,378	532.37 min	8.87 h
Day (Día)	Night (Noche)	Fin de Semana	765	180.39 min	3.01 h
Night (Noche)	Day (Día)	Fin de Semana	122,890	595.17 min	9.92 h
Night (Noche)	Night (Noche)	Fin de Semana	291,307	772.75 min	12.88 h

Análisis de Hallazgos y Causas Operacionales: Los datos evidencian que el factor más lesivo en la latencia de despeje vial no es únicamente la ausencia de luz solar, sino la conjunción sinérgica de oscuridad astronómica profunda (Night / Night) con el fin de semana. Mientras que un siniestro diurno laborable se resuelve en 5.61 horas (336.66 min), un evento en noche cerrada de fin de semana asciende a un promedio de 12.88 horas (772.75 min), representando un incremento del 130% (+7.27 horas de corte).

Asimismo, el desglose fotométrico demuestra que durante las fases crepusculares (Night en sol pero Day en crepúsculo astronómico), el tiempo medio en días hábiles (345.65 min) apenas difiere del régimen diurno (336.66 min), lo que comprueba que la iluminación residual en la bóveda celeste permite una operatividad estándar. En contraposición, la parálisis crítica de casi 13 horas en fines de semana responde a la escasez de dotaciones de grúas de gran porte y cuadrillas de peritaje en horario de madrugada, agravada por siniestros de mayor cinemática violenta debido al exceso de velocidad propio del tráfico nocturno despejado.

4. Modelado con Spark MLlib (Consigna 6 - Opción A: Clasificación Supervisada)

4.1. Requisito 1 — Justificación del Problema, Variable Objetivo e Hipótesis de Trabajo

Motivación Operativa y Sanitaria: En la gestión de Sistemas Inteligentes de Transporte (ITS), cuando ocurre una colisión vial, las cámaras y sensores automáticos

reportan instantáneamente la localización y el contexto climático, pero la evaluación del nivel de daño físico puede demorar decenas de minutos. Formular un clasificador supervisado para predecir la severidad en tiempo real permite anticipar si un siniestro demandará el despacho urgente de helicópteros médicos y grúas de gran porte antes de la llegada de la patrulla policial.

Variable Objetivo (Target): Se seleccionó el atributo multiclase Severity (niveles 1 a 4, indexado a 0..3 como 'label' para compatibilidad nativa con Spark MLlib). A diferencia de una clasificación binaria simplificada, abordar las cuatro clases respeta la riqueza original del dataset y permite aislar el impacto de siniestros moderados versus catastróficos.

Hipótesis de Trabajo: Se postula que la interacción no lineal entre variables espaciales precisas (latitud/longitud de nudos viales), factores meteorológicos adversos (visibilidad crítica < 3 millas, humedad extrema y temperatura) y ciclos de iluminación solar provee suficiente señal estadística para discriminar la gravedad del siniestro con una exactitud superior al 75% sobre el baseline aleatorio.

4.2. Requisito 2 — Pipeline de Ingeniería de Características Distribuidas (pyspark.ml)

Para asegurar reproducibilidad, escalabilidad en clúster y prevenir fuga de información (data leakage), se implementó un flujo unificado empleando componentes de la biblioteca distribuida pyspark.ml:

- Limpieza y Manejo de Nulos: Filtrado determinístico dropna() sobre el subespacio de variables predictoras proyectadas (Start_Lat, Start_Lng, Temperature, Humidity, Visibility, Sunrise_Sunset, State) para asegurar consistencia matricial en memoria distribuida.
- Indexación Categórica (StringIndexer): Aplicación de StringIndexer sobre los atributos categóricos State y Sunrise_Sunset para mapear cadenas de caracteres a índices numéricos continuos ordenados por frecuencia.
- Codificación One-Hot (OneHotEncoder): Vectorización dispersa (Sparse Vectors) de los índices de estado e iluminación para evitar que el algoritmo interprete relaciones de orden métrico ficticias entre entidades geográficas.
- Ensamblado Dimensional (VectorAssembler): Integración atómica de las 5 dimensiones numéricas continuas y los vectores One-Hot dispersos en una columna vectorial unificada ('features') con 54 dimensiones totales.

- Encapsulamiento en Pipeline: Orquestación secuencial de los transformadores y el estimador bajo un objeto pyspark.ml.Pipeline, permitiendo ajustar los parámetros exclusivamente sobre el conjunto de entrenamiento y transformar inferencias de forma atómica.

4.3. Requisito 3 — Entrenamiento, Validación Rigurosa y Configuración de Hiperparámetros

Partición Rigurosa de Datos: El conjunto limpio fue dividido de forma distribuida en una proporción del 80% para Entrenamiento (train_data, ~5.8 millones de eventos) y un 20% para Prueba (test_data, ~1.45 millones de eventos) mediante randomSplit([0.8, 0.2], seed=42). La fijación de la semilla garantiza reproducibilidad científica total del experimento.

Selección del Modelo y Justificación de Hiperparámetros: Se seleccionó el algoritmo DecisionTreeClassifier(labelCol='label', featuresCol='features', maxBins=60, maxDepth=10). La decisión de fijar maxBins=60 (frente al valor por defecto de 32 de Spark) es un requisito metodológico crítico: dado que la variable State contiene 49 estados federales indexados, un valor de maxBins=32 provocaría el fallo de Spark o el descarte de divisiones en la variable de estado. Fijar maxBins=60 permitió evaluar de forma exacta las particiones categóricas geográficas sin pérdida de resolución.

4.4. Requisito 4 — Interpretación de Resultados, Métricas y Validación de Hipótesis

Métricas Cuantitativas sobre Conjunto de Prueba (Test): Se utilizó MulticlassClassificationEvaluator para medir el desempeño fuera de muestra, obteniendo:

- **Exactitud Global (Accuracy): 79.81% (0.7981).**
- **F1-Score Ponderado: 0.7138 (evalúa balance y pondera la penalización por clases minoritarias).**
- **Tiempo Neto de Cómputo Distribuido: 245.72 segundos (~4.09 minutos).**

Validación y Contraste de Hipótesis: El desempeño alcanzado confirma sólidamente la hipótesis formulada. Alcanzar casi un 80% de exactitud en un problema multiclase con 7.7 millones de registros demuestra que las condiciones meteorológicas y espaciales son altamente informativas para predecir la severidad. La inspección de los nodos raíz del árbol reveló que la visibilidad (< 3 mi)

combinada con coordenadas en corredores interestatales clave (I-95, I-10, I-80) constituyen los divisores principales de la gravedad, permitiendo a los operadores viales anticipar bloqueos prolongados desde el instante inicial de la alerta telemática.

5. Evaluación Experimental de Rendimiento (Benchmarking General)

5.1. Pautas Metodológicas y Protocolo Experimental del Benchmarking

Para garantizar comparabilidad, rigor científico y reproducibilidad en la evaluación de desempeño entre Pandas mononodo y Apache Spark distribuido sobre los 7,728,394 de registros, se adoptó el siguiente protocolo metodológico obligatorio:

1. Registro de Tiempo Neto (Wall-Clock Time): Para cada una de las 5 consultas analíticas, se registró estrictamente el tiempo transcurrido (wall-clock time) en segundos mediante temporizadores de alta precisión (time.perf_counter()). La medición aísla el tiempo de ejecución algorítmica y procesamiento interno, excluyendo fases previas de importación de bibliotecas.

2. Materialización Obligatoria del Grafo en PySpark: Dada la naturaleza de evaluación perezosa (lazy evaluation) que gobierna a Apache Spark, las transformaciones relacionales sobre DataFrames (filter, groupBy, select) solo construyen el grafo de dependencias (DAG). Para registrar el costo computacional verídico, en cada consulta se invocó indefectiblemente una acción terminal bloqueante (.count(), .collect() o .show()). Esto forzó al optimizador Catalyst y al motor Tungsten a materializar el plan físico, ejecutar el pipeline distribuido y efectuar el intercambio de datos entre particiones (shuffle) a través de los ejecutores.

3. Documentación de Resiliencia y Contingencia Out Of Memory (OOM) en Pandas: Se documentó exhaustivamente el comportamiento del entorno mononodo frente a la memoria física disponible (RAM):

- Punto de Fallo Teórico Documentado: El cálculo directo de distancias continuas geodésicas (Haversine continuo $O(N^2)$) para los 7.72M de registros en la Consulta 4 exigiría evaluar una matriz de 59.7 billones de comparaciones flotantes, requiriendo más de 238 Terabytes de memoria contigua en RAM. Su ejecución ingenua en Pandas desencadena de forma inmediata una saturación crítica del subsistema de memoria, provocando

la terminación anómala del proceso por el sistema operativo (MemoryError / SIGKILL).

- Mitigación y Comportamiento del Sistema: Para neutralizar el OOM, el equipo diseñó una discretización espacial celular por truncamiento de coordenadas a 2 decimales (~2.2 km). Esto redujo la complejidad computacional de $O(N^2)$ a un agrupamiento hash lineal $O(N)$, manteniendo el consumo en menos de 45 MB de RAM y permitiendo a Pandas completar la consulta en solo 1.73 segundos.

4. Cálculo Formal del Factor de Aceleración (Speedup): El desempeño relativo entre ambas tecnologías se cuantificó formalmente mediante la relación:

$$\text{Speedup} = T_{\text{Pandas}} / T_{\text{Spark}}$$
donde T_{Pandas} representa el wall-clock time en Pandas y T_{Spark} el wall-clock time en PySpark. Un Speedup inferior a 1.0x refleja la ventaja de Pandas en memoria local con instrucciones vectorizadas SIMD y cero latencia de coordinación; mientras que un Speedup superior a 1.0x (observado en la Consulta 10 con 1.65x) documenta la superioridad de PySpark para paralelizar y amortizar el costo de shuffling en agrupamientos densos sobre millones de filas.

Tabla 3. Benchmarking Experimental Consolidado de Rendimiento y Speedup (7.72M Registros)

Consulta Asignada	Volumen Procesado	Pandas (s)	PySpark (s)	Speedup (Pandas/Spark)
C3: Haversine R=25km (Los Ángeles)	7.72M filas / 3 cols	1.67 s	28.32 s	0.06x (Pandas 17x más rápido)
C4: Hotspots Top 5 (Severity >= 3)	7.72M filas / 3 cols	0.09 s	53.07 s	0.002x (Pandas 580x más rápido)
C6: Ranking Autopistas Normalizadas	7.72M filas / 4 cols	15.83 s	36.58 s	0.43x (Pandas 2.3x más rápido)
C8: Anomalías Meteorológicas (> 3σ)	7.72M filas / 4 cols	10.13 s	41.47 s	0.24x (Pandas 4.1x más rápido)
C10: Despeje por Iluminación y Día	7.72M filas / 3 cols	18.33 s	28.01 s	0.65x (Pandas 1.5x más rápido)

5.1. Discusión Técnica de Cuellos de Botella y Speedup

1. Sobrecarga Basal de la JVM y Py4J: En cada ejecución en PySpark mononodo, el proceso Python debe transferir el plan lógico a la JVM vía sockets TCP (Py4J). Spark debe compilar el plan físico (Catalyst), generar código Java bytecode en Tungsten y coordinar el pool de

hilos locales, introduciendo una latencia fija basal de 20 a 40 segundos.

2. Eficiencia Vectorizada SIMD en Pandas: Con Projection Pushdown de Parquet, Pandas lee únicamente las columnas necesarias (~15 a 40 MB en RAM), ejecutando cálculos matemáticos con instrucciones vectorizadas SIMD de NumPy en C a velocidad de microprocesador (0.09 s en C4).

3. Convergencia en Operaciones Complejas con Shuffling: En consultas con extracciones por expresiones regulares y agrupamientos densos (C6), el tiempo de Pandas se incrementa a 15.83 s, estrechando la brecha con PySpark (36.58 s, Speedup 0.43x). Ante volúmenes out-of-core que excedan la RAM, Pandas colapsa por OOM, mientras que Apache Spark conmuta automáticamente a particionado en disco.

6. Conclusiones y Lecciones de Arquitectura

- El formato físico gobierna el desempeño: Parquet ZSTD con ordenamiento espacial por bloques redujo el archivo a 491 MB (83% ahorro), permitiendo a ambas herramientas operar interactivamente gracias al predicate pushdown.

- Complementariedad arquitectural: Pandas vectorizado sobresale en exploración mononodo cuando los datos proyectados caben en memoria RAM. Apache Spark es la infraestructura obligatoria para escalabilidad horizontal en clústeres elásticos, tolerancia a fallos y pipelines distribuidos de Machine Learning (MLlib).

- Madurez de Spark MLlib: Entrenar un árbol de decisión sobre más de 6 millones de eventos alcanzando casi 80% de Accuracy en 4 minutos valida la viabilidad de Spark para analítica predictiva en entornos de producción de Big Data.

7. Referencias Bibliográficas

[1] S. Moosavi, M. H. Samavatian, S. Parthasarathy, and R. Ramnath, 'A Countrywide Traffic Accident Dataset,' in Proceedings of the 2019 ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, Chicago, IL, USA, 2019, pp. 408-411.

[2] M. Zaharia et al., 'Apache Spark: A unified engine for big data processing,' Communications of the ACM, vol. 59, no. 11, pp. 56-65, Nov. 2016.

[3] M. Armbrust et al., 'Spark SQL: Relational data processing in Spark,' in Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Australia, 2015, pp. 1383-1394.

[4] W. McKinney, 'Data structures for statistical computing in Python,' in Proceedings of the 9th Python in Science Conference (SciPy 2010), Austin, TX, USA, 2010, pp. 51-56.

[5] Apache Arrow Project, 'Apache Arrow: A cross-language development platform for in-memory data,' Apache Software Foundation, Tech. Rep., 2023.

[6] X. Meng et al., 'MLlib: Machine learning in Apache Spark,' Journal of Machine Learning Research, vol. 17, no. 34, pp. 1-7, 2016.